

Who may do what?



Write the rules: anyone logged in reads, only the owner changes, only staff delete.

The big idea

"May this request go ahead?" is two questions.

Authentication: who are you? Django checks the proof and stores the user in `request.user`.

Permissions: are you allowed? A permission class looks at the user, the method and the object, and answers `True` or `False`.

What your methods receive

No real web request: the tests build fake ones and call your methods directly.

```
owner = User.objects.create_user("owner", password="x")
other = User.objects.create_user("other", password="x")
staff = User.objects.create_user("staff", password="x", is_staff=True)
note = SimpleNamespace(owner=owner)           # a pretend object

request = RequestFactory().patch("/notes/1/") # a fake PATCH request
request.user = other                          # made by "other"

IsOwnerOrReadOnly().has_permission(request, None) # view is None
IsOwnerOrReadOnly().has_object_permission(request, None, note)
```

What the tests expect back

Must say yes (True)

- Logged in: `has_permission` (test 2)
- Anyone reads the note (test 3)
- `owner` changes the note (test 4)
- Not `DELETE`: `IsStaffForDelete` (test 6)
- `staff` sends `DELETE` (test 7)

Must say no (False)

- Anonymous: `has_permission` (test 1)
- `other` changes the note (test 5)
- Not staff, sends `DELETE` (test 7)

New words

request.user

The user making the request, filled in before your code runs.

AnonymousUser

What **request.user** is when nobody logged in.

is_authenticated

True for a logged-in user, **False** for **AnonymousUser**.

is_staff

True for team members (admins), **False** for normal users.

owner

The user an object belongs to, like **note.owner**.

safe method

Only reads: **GET**, **HEAD**, **OPTIONS**. Others change data.

401 / 403

"Log in first" / "logged in, but not allowed".

Tool: a permission class

Up to two methods. In a real API, DRF calls them for you; `False` stops the request.

```
class SomePermission(BasePermission):
    def has_permission(self, request, view):
        # EVERY request, first: may this user use this endpoint?
        return True

    def has_object_permission(self, request, view, obj):
        # Only for ONE object, like DELETE /notes/1/, after it's loaded:
        # may this user do this to THIS object?
        return True
```

Tool: user attributes (no brackets)

```
from django.contrib.auth.models import AnonymousUser, User

ada = User.objects.create_user("ada", password="x")
boss = User.objects.create_user("boss", password="x", is_staff=True)
visitor = AnonymousUser()
print(ada.is_authenticated, ada.is_staff)
print(boss.is_authenticated, boss.is_staff)
print(visitor.is_authenticated, visitor.is_staff)
```

OUTPUT

```
True False
True True
False False
```

Tool: `SAFE_METHODS` and `in`

`x in something` checks whether `x` is one of the items.

```
print(SAFE_METHODS)
print("GET" in SAFE_METHODS)
print("PATCH" in SAFE_METHODS)
```

OUTPUT

```
('GET', 'HEAD', 'OPTIONS')
True
False
```

Tool: comparing users with ==

```
from types import SimpleNamespace
from django.contrib.auth.models import User

ada = User.objects.create_user("ada", password="x")
grace = User.objects.create_user("grace", password="x")
recipe = SimpleNamespace(title="Pancakes", owner=ada)
print(recipe.owner == ada)
print(recipe.owner == grace)
print(recipe.owner == User.objects.get(username="ada"))
```

OUTPUT

```
True
False
True
```

has_permission vs has_object_permission

has_permission(request, view)

- Runs for **every** request, first
- No object: it can't see an owner
- Good for: logged in? is it a **DELETE**?
- Tests 1-2, 6-7

has_object_permission(..., obj)

- Runs only for **one** object, like `/notes/1/`
- Gets `obj`, so it can check `obj.owner`
- Good for: is this user the owner?
- Tests 3-5

Pattern: answer early, otherwise decide

The same shape as `has_object_permission`, on different data:

```
def can_enter(age, has_ticket):  
    if age < 5:  
        return True          # small children always get in  
    return has_ticket        # everyone else needs a ticket  
  
print(can_enter(3, False), can_enter(30, True), can_enter(30, False))
```

OUTPUT

True True False

Your task, step by step

- 1 `has_permission: return request.user.is_authenticated` → tests 1-2
- 2 `has_object_permission: method in SAFE_METHODS? Return True` → test 3
- 3 Otherwise return whether `obj.owner` is the same user as `request.user` → tests 4-5
- 4 `IsStaffForDelete: "DELETE" → request.user.is_staff, else True` → tests 6-7

Common mistakes

- `is_authenticated()` with brackets: `TypeError`, it's an attribute
- Checking the owner in `has_permission`: there's no `obj` there
- A path with no `return` gives `None`, which counts as no
- `request.method == "delete"` is never true: it's `"DELETE"`
- Compare users, `obj.owner == request.user`, not their usernames

Where you'll use this

A notes app: everyone on the team can read every note, but only you can edit yours, and only an admin can delete anything.

- In a DRF view: `permission_classes = [IsOwnerOrReadOnly, IsStaffForDelete]`
- DRF calls your methods on every request and replies `401` or `403` when they say no
- The view's code never runs for a refused request

Check yourself

1. What's the difference between authentication and permissions?
2. Why must the owner check go in `has_object_permission`?
3. Why write `is_authenticated` without brackets?
4. An anonymous user sends `DELETE`. Why don't you need a separate check?

Keep learning

[HTTP 401 Error vs HTTP 403 Error – Status Code Responses Explained](#)

freeCodeCamp · article · beginner. Start here: the difference between 'who are you?' (401) and 'you may not do that' (403), which permissions decide.

[Tutorial 4: Authentication & Permissions](#)

Django REST framework · tutorial · beginner. Builds an IsOwnerOrReadOnly permission step by step, almost the same class you write here.

[Permissions: custom permissions](#)

Django REST framework · docs · intermediate. Official reference for BasePermission, has_permission, has_object_permission and SAFE_METHODS.

[Authorization Cheat Sheet](#)

OWASP · article · intermediate. Security rules of thumb such as 'deny by default' and 'check on every request', to read after the exercise.